

Machine Learning

Winter 2011/12

Roland Memisevic

Lecture 12, Jan. 30, 2012

Representing data

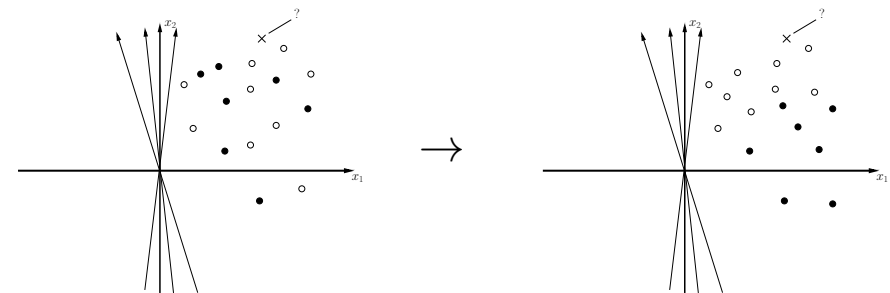


- ▶ Consider the binary problem: **Car** or **No Car**?
- ▶ Which representation would make this easier to learn:
 1. Images represented using gray-values:
[1.pixel :120.0, 2.pixel :108.0, 3.pixel :103.0, ...]
 2. Images represented using higher-level concepts:
[Wheels :2, Buildings :1, Tree leaves :737, ...]

Outline

- ▶ Learning to represent data
- ▶ Feature learning and sparse coding
- ▶ Restricted Boltzmann machines

Better representations can simplify learning



- ▶ Representing data in the right way can give us an input space where learning becomes much easier.

Representing data

- ▶ Detecting, say, wheels in the example may pose a whole lot of extra problems.
- ▶ But there can be a middle-ground: Simpler, but still useful, features, would be, for example configurations of edges, blobs or other primitives in the image.
- ▶ This is something we can learn from the data.
- ▶ Learning to (re-)represent data is, of course, a form of unsupervised learning.
- ▶ Thinking about unsupervised learning as a way to get better features suggests a variety of extensions and provides some interesting new insights.

Learning feature hierarchies

- ▶ The structure of real world data is often hierarchical:
- ▶ Images are composed of objects (eg., cars, houses), which are composed of parts (wheels, doors, windows), which are composed of sub-parts (bars, circles, edges), etc.
- ▶ This means, we are likely to do better if we learn *feature hierarchies* rather than just features.
- ▶ One way to do this is *greedily*: First learn features; Then learn combinations of features; Then combinations of combinations; Etc.
- ▶ This is called “*Deep Learning*”: Given a module that can extract features from data, apply the module recursively, by stacking multiple copies on top of each other.
- ▶ This will only make sense, if the feature extraction module is a *non-linear* function of its inputs.

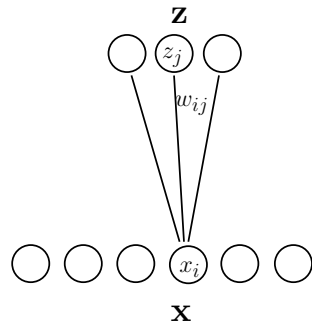
Real world data is not “random”

- ▶ Data we deal with is usually highly structured because it was produced in the real world and according to the laws of physics.
- ▶ For example, the set of real-world images of size 100×100 pixels (“all photos that all humans will ever take”) is much smaller than the set of *all possible* images of size 100×100 pixels.
- ▶ Since the real world gives rise to both the inputs and the labels, it seems reasonable to learn about the real world first, even if the ultimate goal is to map inputs to labels.
- ▶ Learning of good representations is also known as **feature learning**.

Semi-supervised learning

- ▶ Learning features can also help in situations where we have a lot of unlabeled data, but not much labeled data.
- ▶ Utilizing unlabeled data in a supervised learning problem is known as **semi-supervised learning**.
- ▶ This is a common problem, because labeling can be expensive!
- ▶ For example, there are *a lot* of unlabeled images on the web, but telling which one contains a car is tedious and time-consuming.

Unsupervised learning modules

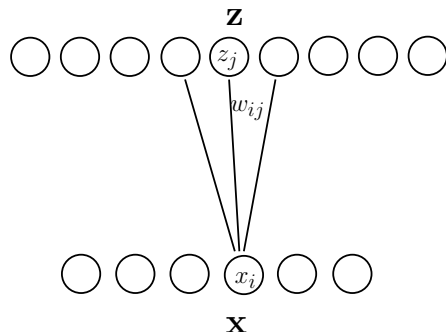


- ▶ Most unsupervised learning methods can be represented as a bi-partite network.
- ▶ Example: PCA learns linear mappings.
- ▶ Example: MoG learns linear mappings followed by a “softmax” non-linearity.
- ▶ More examples: K-means, auto-encoders, etc.

No manifold, no clusters

- ▶ PCA and its non-linear extensions look for a single, homogenous manifold in the data.
- ▶ Clustering, in contrast, finds convex “blobs”.
- ▶ Often, neither assumption is correct, and data is structured in more complicated ways.
- ▶ For example, we may have a manifold with different dimensionalities in different parts of the data-space.
- ▶ Or we may have clusters that look more like pieces of a manifold than like convex blobs.
- ▶ We can learn this type of structure by using an *overcomplete* latent representation (more latent dimensions than input dimensions):

Feature learning



- ▶ An overcomplete z will be able to reconstruct the data trivially, and it will overfit.
- ▶ So we have to constrain the overcomplete z in some way.
- ▶ The common way to do this is to force most of the latent components, z_k , to be zero.
- ▶ This is known as *sparse coding*.

Sparse coding benefits

- ▶ Subsets of hidden variables can now take care of different areas, or “sub-manifolds”, in data-space.
- ▶ Both, (i) which hidden units are non-zero, and (ii) the values of those hidden units, will be a function of the input-data.
- ▶ That way, we can learn more complex structure than a manifold or a set of convex blobs.
- ▶ An additional benefit of sparse codes: They can represent a *collection* (or *set*) of vectors as a single vector as follows. Represent each element by a sparse vector, and let their sum or mean represent the set. The sum of sparse vectors contains few “collisions”, so the sum will not lose much information.

Sparse coding methods

- ▶ There are a lot of ways to learn overcomplete representations. Common ones are:
 1. Minimize reconstruction error like PCA, but add a *sparsity penalty* (for example, like Lasso) for $\mathbf{z}$:

$$\|\mathbf{x} - \mathbf{W}^T \mathbf{z}\|^2 + \lambda \|\mathbf{z}\|_1$$

2. “K-SVD”
 3. Independent components analysis (ICA)
 4. Restricted Boltzmann machine (RBM)
 5. De-noising auto-encoder, contractive auto-encoder
- ▶ 4. and 5. were not introduced specifically for learning sparse representations, but they can do so very well. More importantly: they are non-linear, so we can stack them.
 - ▶ Learning of over-complete representations is also called “dictionary learning” or just “feature learning”.

Restricted Boltzmann machines

- ▶ The Restricted Boltzmann Machine (RBM) is a feature learning method that defines the joint over data $\mathbf{x}$ and hidden variables $\mathbf{z}$.
- ▶ It is a Boltzmann machine without connections between hidden or between observables.
- ▶ Since, unlike many unsupervised learning methods (PCA, MoG, etc.), RBMs parameterize the *joint* $p(\mathbf{x}, \mathbf{z})$ directly and not the decomposition $p(\mathbf{x}|\mathbf{z})p(\mathbf{z})$, they do not need an explicit prior $p(\mathbf{z})$.
- ▶ Since for every joint, $p(\mathbf{x}, \mathbf{z}) = p(\mathbf{x}|\mathbf{z})p(\mathbf{z})$ is true, the RBM will define a prior *implicitly*. But we usually don't care what it is.

Boltzmann machines

- ▶ The RBM is a probabilistic feature learning method, and it is special case of a *Boltzmann machine*:
- ▶ A Boltzmann machine models the distribution over binary data and/or hidden $\mathbf{x}$, via an *energy function*

$$E(\mathbf{x}) = - \sum_{ij} W_{ij} x_i x_j - \sum_i c_i x_i$$

which assigns small values to points we want to have high probability and vice versa. $\mathbf{W}$ and $\mathbf{c}$ are parameters.

- ▶ The probability of $\mathbf{x}$ is then defined as

$$p(\mathbf{x}) = \frac{1}{Z} \exp(-E(\mathbf{x})), \quad Z = \sum_{\mathbf{x}} \exp(-E(\mathbf{x}))$$

- ▶ This is an undirected graphical model with potential functions $\psi(x_i, x_j) = W_{ij} x_i x_j$ and $\psi(x_i) = \exp(c_i x_i)$, and Z is the partition function.
- ▶ The model can be extended to deal with real-valued data.

Binary Restricted Boltzmann Machines definition

RBM for binary data

RBMs define joint over data $\mathbf{x}$ and latent variables $\mathbf{z}$ using the energy function:

$$E(\mathbf{x}, \mathbf{z}) = - \sum_{i=1}^D \sum_{j=1}^K W_{ij} x_i z_j - \sum_j b_j z_j - \sum_i c_i x_i$$

which we exponentiate and normalize to get the distribution:

$$\begin{aligned} p(\mathbf{x}, \mathbf{z}) &= \frac{1}{Z} \exp(-E(\mathbf{x}, \mathbf{z})) \\ &= \frac{1}{Z} \exp\left(\sum_{ij} W_{ij} x_i z_j + \sum_j b_j z_j + \sum_i c_i x_i\right) \end{aligned}$$

where $Z = \sum_{\mathbf{z}, \mathbf{x}} \exp(-E(\mathbf{x}, \mathbf{z}))$

Binary RBM marginals

- The log-probability of a data point, $\mathbf{x}$, is:

$$\begin{aligned}
 \log p(\mathbf{x}) &= \log \sum_{\mathbf{z}} p(\mathbf{x}, \mathbf{z}) \\
 &= -\log Z + \log \sum_{\mathbf{z}} \exp \left(\sum_{ij} W_{ij} x_i z_j + \sum_j b_j z_j + \sum_i c_i x_i \right) \\
 &= -\log Z + \log \sum_{z_1} \dots \sum_{z_K} \exp \left(\sum_j z_j [b_j + \sum_i W_{ij} x_i] \right) \exp \left(\sum_i c_i x_i \right) \\
 &= -\log Z + \log \sum_{z_1} \dots \sum_{z_K} \prod_j \exp \left(z_j [b_j + \sum_i W_{ij} x_i] \right) \exp \left(\sum_i c_i x_i \right) \\
 &= -\log Z + \log \sum_{z_1} \exp \left(z_1 [b_1 + \sum_i W_{i1} x_i] \right) \sum_{z_2} \exp \left(\dots \right) \sum_{z_3} \dots \\
 &\quad \dots \sum_{z_K} \exp \left(z_K [b_K + \sum_i W_{iK} x_i] \right) + \sum_i c_i x_i \\
 &= -\log Z + \sum_j \log \left(1 + \exp \left(b_j + \sum_i W_{ij} x_i \right) \right) + \sum_i c_i x_i
 \end{aligned}$$

- The probability, accordingly, is proportional to a product over K terms of the form $(1 + \exp(\dots))$:

$$p(\mathbf{x}) \propto \prod_j \left(1 + \exp \left(b_j + \sum_i W_{ij} x_i \right) \right)$$

Computing $p(\mathbf{x})$

- Computing the probability is, in general, not tractable, because we cannot compute $(\log)Z$.
- But we can *compare* the (log-) probability for two points $\mathbf{x}_1, \mathbf{x}_2$ because $(\log)Z$ will cancel in the ratio (difference).
- So we can always tell, which point is more likely under the model. In many applications, this is all we need.
- And in other applications we just need to compute the conditional $p(\mathbf{z}|\mathbf{x})$ which allows us to infer the hidden codes given new data $\mathbf{x}$. This turns out to be easy and efficient in the RBM:

Binary RBM marginals and free energy

- It is often convenient to write the marginal alternatively as follows:

$$p(\mathbf{x}) = \frac{1}{Z} \exp \left(-\mathcal{F}(\mathbf{x}) \right)$$

where we define the “Free Energy”

$$\mathcal{F}(\mathbf{x}) = -\log \sum_{\mathbf{z}} \exp \left(-E(\mathbf{x}, \mathbf{z}) \right)$$

and Z is now $\sum_{\mathbf{x}} \exp(-\mathcal{F}(\mathbf{x}))$.

- Now $p(\mathbf{x})$ takes the same form as a fully observed Boltzmann machine without hidden.

Binary RBM conditionals over hidden

- The conditional over the hidden variables, given data, is

$$\begin{aligned}
 p(\mathbf{z}|\mathbf{x}) &\propto p(\mathbf{x}, \mathbf{z}) = \Omega_1 p(\mathbf{x}, \mathbf{z}) \\
 &= \Omega_1 \exp \left(\sum_{ij} W_{ij} x_i z_j + \sum_j b_j z_j + \sum_i c_i x_i \right) \\
 &= \Omega_1 \exp \left(\sum_j z_j [b_j + \sum_i W_{ij} x_i] \right) \exp \left(\sum_i c_i x_i \right) \\
 &= \Omega_2 \exp \left(\sum_j z_j [b_j + \sum_i W_{ij} x_i] \right) \\
 &= \Omega_2 \prod_j \exp \left(z_j [b_j + \sum_i W_{ij} x_i] \right)
 \end{aligned}$$

- This is a product over all z_j , so the hidden are independent, given the data.
- For each z_j we have $p(z_j|\mathbf{x}) \propto \exp \left(z_j [b_j + \sum_i W_{ij} x_i] \right)$
- But z_j is binary, so we have

$$p(z_j = 1) = \frac{\exp(b_j + \sum_i W_{ij} x_i)}{1 + \exp(b_j + \sum_i W_{ij} x_i)} = \frac{1}{1 + \exp(-[b_j + \sum_i W_{ij} x_i])}$$

- This is just the logistic sigmoid applied to a linear function of the data!

Binary RBM conditionals over data

- By using an analogous calculation for $\mathbf{x}$ we get:

$$p(x_i = 1|\mathbf{z}) = \frac{1}{1 + \exp(-[c_i + \sum_j W_{ij} z_j])}$$

- Furthermore, it turns out that if we change the RBM energy function to

$$E(\mathbf{x}, \mathbf{z}) = - \sum_{i=1}^D \sum_{j=1}^K W_{ij} x_i z_j - \sum_j b_j z_j - \sum_i c_i x_i + \frac{1}{2\sigma^2} \sum_j (y_j - w_j)^2$$

the conditional over $\mathbf{x}$ is a Gaussian with mean

$$c_i + \sum_j W_{ij} z_j$$

and variance σ^2 in each dimension.

- Similarly, one can turn the conditionals over data or hiddens into any *exponential family* distribution.

Binary RBM derivatives

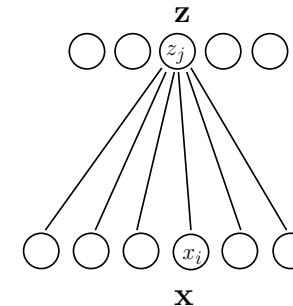
- To train the model we need the derivative of the log-probability of data.
- Let θ denote some parameter in the model. We have

$$\begin{aligned} \frac{\partial \log p(\mathbf{x})}{\partial \theta} &= -\frac{\partial \mathcal{F}(\mathbf{x})}{\partial \theta} + \frac{1}{Z} \sum_{\mathbf{x}} \exp(-\mathcal{F}(\mathbf{x})) \frac{\partial \mathcal{F}(\mathbf{x})}{\partial \theta} \\ &= -\frac{\partial \mathcal{F}(\mathbf{x})}{\partial \theta} + \sum_{\mathbf{x}} p(\mathbf{x}) \frac{\partial \mathcal{F}(\mathbf{x})}{\partial \theta} \end{aligned}$$

- Furthermore, we have

$$\begin{aligned} \frac{\partial \mathcal{F}(\mathbf{x})}{\partial \theta} &= \sum_{\mathbf{z}} \frac{\exp(-E(\mathbf{x}, \mathbf{z}))}{\sum_{\mathbf{z}'} \exp(-E(\mathbf{x}, \mathbf{z}'))} \frac{\partial E(\mathbf{x}, \mathbf{z})}{\partial \theta} \\ &= \sum_{\mathbf{z}} p(\mathbf{z}|\mathbf{x}) \frac{\partial E(\mathbf{x}, \mathbf{z})}{\partial \theta} \end{aligned}$$

RBM as undirected graphical model



- Since the RBM is an undirected graphical model, a simpler way to derive the conditional independence properties is by looking at the graph.

Binary RBM derivatives

- By putting everything together we get:

$$\frac{\partial \log p(\mathbf{x})}{\partial \theta} = - \sum_{\mathbf{z}} p(\mathbf{z}|\mathbf{x}) \frac{\partial E(\mathbf{x}, \mathbf{z})}{\partial \theta} + \sum_{\mathbf{x}', \mathbf{z}} p(\mathbf{x}', \mathbf{z}) \frac{\partial E(\mathbf{x}', \mathbf{z})}{\partial \theta}$$

with

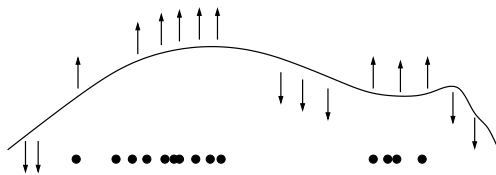
$$\frac{\partial E(\mathbf{x}, \mathbf{z})}{\partial W_{ij}} = -x_i z_j, \quad \frac{\partial E(\mathbf{x}, \mathbf{z})}{\partial b_j} = -z_j, \quad \frac{\partial E(\mathbf{x}, \mathbf{z})}{\partial c_i} = -x_i$$

- This is the sum of two terms: (i) an expectation under $p(\mathbf{z}|\mathbf{x})$ and (ii) an expectation under the model distribution $p(\mathbf{x}, \mathbf{z})$.
- The second term is, in general, is not tractable, but we can approximate it using Gibbs sampling:

RBM Gibbs sampling

- ▶ Because of the conditional independencies, it is easy to sample from $p(\mathbf{z}|\mathbf{x})$ and from $p(\mathbf{x}|\mathbf{z})$.
- ▶ So to perform Gibbs sampling, we just need to iterate sampling $\mathbf{z}$ from $p(\mathbf{z}|\mathbf{x})$ and $\mathbf{x}$ from $p(\mathbf{x}|\mathbf{z})$.
- ▶ In the limit we will approximate sampling from the joint $p(\mathbf{x}, \mathbf{z})$.
- ▶ Using L samples $(\mathbf{x}^{(l)}, \mathbf{z}^{(l)})$ from the joint we can then approximate the second term using $\frac{1}{L} \sum_{l=1}^L \frac{\partial E(\mathbf{x}^{(l)}, \mathbf{z}^{(l)})}{\partial \theta}$.
- ▶ For learning, we iterate parameter updates and sampling.

Learning energy models



- ▶ Maximum likelihood training has the effect of increasing $p(\mathbf{x})$ at the data, and decreasing $p(\mathbf{x})$ everywhere else.

RBM learning intuition

- ▶ Sampling from the model will produce “fantasy data” according to the model distribution.
- ▶ Since the fantasy data will enter the update equation with a negative sign, learning has the effect of increasing the probability where the training data is, and reducing it proportional to where the model *thinks* it should be.
- ▶ (The same is actually true in all exp(energy)-type models, including logistic regression).
- ▶ The weight update for the first term in the derivative is sometimes called “**positive phase**”. It is easy and efficient, because it involves only the conditional $p(\mathbf{z}|\mathbf{x})$.
- ▶ The update for the negative term is sometimes called “**negative phase**”, and it involves prolonged sampling to reach the equilibrium joint distribution $p(\mathbf{x}, \mathbf{z})$.

Contrastive divergence training

- ▶ This intuition suggests a short-cut to speed up the sampling, consisting of two modifications to standard Gibbs sampling:
 1. Start sampling at the data.
 2. Rather than waiting to reach the equilibrium, sample for a few steps only (for example, for 1 step).
- ▶ This is known as *contrastive divergence* (CD) learning.
- ▶ For one-step CD, the positive phase consists in computing, or sampling from, $p(\mathbf{z}|\mathbf{x})$ and the negative phase of sampling from $p(\mathbf{x}|\mathbf{z})$.
- ▶ The result is a model that is a good *near the data* and potentially bad elsewhere in the data space.
- ▶ (Whether you really need a model that assigns low probability to points that look nothing like the data will depend on the application.)

Encouraging sparsity

- ▶ There are various ways to encourage hidden variables to be extra-sparse.
- ▶ Most of these consist in adding a penalty function to the learning objective, such as

$$-\lambda \sum_{j=1}^K \left(B - \frac{1}{N} \sum_{n=1}^N p(z_j | \mathbf{x}_n) \right)^2$$

to pull hidden variable activities towards B during training.

- ▶ B is a small positive number, for example, $B = 0.1$.

Energy based learning

- ▶ The view of learning as decreasing energy at data and increasing energy elsewhere can be quite liberating:
- ▶ We may even drop the notion of probabilities altogether, and view learning as “carving out an energy landscape” in data space.
- ▶ Inference then amounts to climbing down the landscape.
- ▶ This works for conditional models, like regression and classification, too: Define the energy landscape over variables $\mathbf{y}$ as a function of other variables $\mathbf{x}$.
- ▶ A good introduction is LeCun et al. (2006) “A Tutorial on Energy-Based Learning”